

Introduction to Linux / Unix

A Complete Beginner's Guide

Using WSL2 (Linux on Windows 11)

Designed for Freshers & Beginners

Topics Covered

1. Introduction to Unix / Linux
2. Working with Files and Directories
3. Permissions and Ownership
4. Processes and System Information
 5. Redirection and Pipes
 6. Shell Scripting Basics

1. Introduction to Unix / Linux

Unix is a family of operating systems created in 1969 at AT&T Bell Labs by Ken Thompson and Dennis Ritchie. It became the foundation for modern operating systems. Linux, created by Linus Torvalds in 1991, is a free and open-source Unix-like operating system that today powers Android phones, web servers, cloud platforms, and supercomputers.

1.1 Why Learn Linux?

Linux is everywhere in the tech industry. Understanding it is essential for software development, DevOps, cloud computing, cybersecurity, and data science. Here are the key reasons to learn it:

- Free and open-source — no licensing fees
- Powers over 96% of the world's top web servers
- Foundation of Android and most cloud infrastructure
- Preferred environment for programming and scripting
- Strong job market demand — employers expect Linux skills

1.2 Setting Up Linux on Windows 11 with WSL2

WSL2 (Windows Subsystem for Linux 2) lets you run a real Linux kernel directly on Windows 11 — no dual-boot, no virtual machine needed. It is the recommended way to learn Linux on a Windows laptop.

Step-by-step WSL2 installation:

- Open PowerShell as Administrator (right-click Start > Windows Terminal (Admin))
- Run the installation command:

```
wsl --install
```

- Restart your computer when prompted
- After restart, Ubuntu opens automatically — set your Linux username and password
- Install Windows Terminal from Microsoft Store for a better experience (tabs, colors, split panes)

TIP

Your Windows files are accessible inside Linux at `/mnt/c/Users/YourName` — so you can work on the same files from both systems!

1.3 The Unix Philosophy

Unix was designed around a simple but powerful philosophy that makes it elegant and efficient:

Command	Description
<code>Everything is a file</code>	Devices, network sockets, directories — all treated as files

Command	Description
One tool, one job	Each command does one thing, and does it well
Chain tools together	Combine simple tools with pipes to solve complex problems
Plain text is universal	Data flows between tools as readable text streams
Case-sensitive	file.txt and File.txt are completely different files

1.4 Your First Commands

Type these commands in your Ubuntu / WSL terminal. The \$ symbol is the prompt — you do not type it.

```
$ whoami           # Shows your current username
$ hostname         # Shows the machine name
$ date            # Shows current date and time
$ uname -a        # Shows detailed OS information
$ echo "Hello Linux!" # Prints a message to the screen
$ clear           # Clears the terminal screen
```

TIP Press the Tab key to auto-complete commands and filenames. Press the Up arrow to recall previous commands. These two shortcuts save enormous amounts of time!

1.5 Getting Help

Linux has a built-in help system. You never need to Google a command if you know these:

Command	Description
<code>man ls</code>	Opens the full manual (man page) for the ls command. Press Q to quit.
<code>ls --help</code>	Prints a short help summary for most commands
<code>whatis ls</code>	Prints a one-line description of what a command does
<code>apropos copy</code>	Searches all man pages for commands related to 'copy'

2. Working with Files and Directories

The Linux filesystem is a single tree that starts at the root directory `/`. Everything — files, programs, hardware devices — lives somewhere in this tree. Understanding the layout is the first step to navigating confidently.

2.1 The Linux Filesystem Layout

<code>/</code>	← Root of everything
<code> — home/</code>	← All user home directories
<code> — john/</code>	← Your home folder (same as <code>~</code>)
<code> — etc/</code>	← System configuration files
<code> — bin/</code>	← Essential programs (<code>ls</code> , <code>cp</code> ,
<code>mkdir...</code>	
<code> — usr/</code>	← User-installed software
<code> — var/</code>	← Log files and variable data
<code> — tmp/</code>	← Temporary files (cleared on reboot)
<code> — dev/</code>	← Device files (disks, USB, terminal)
<code> — mnt/c/</code>	← Your Windows C: drive (WSL only)

TIP In WSL, `/mnt/c` corresponds to your Windows C:\ drive. You can open your Windows Desktop with: `cd /mnt/c/Users/YourName/Desktop`

2.2 Navigating the Filesystem

Navigation is done with three essential commands: `pwd` (where am I?), `ls` (what is here?), and `cd` (go somewhere).

```
$ pwd                                # Print Working Directory — shows
your current location
/home/john
```

```
$ ls                                # List files in current directory
Desktop  Documents  Downloads  Pictures
```

```
$ ls -l                             # Long listing — shows permissions,
size, date
drwxr-xr-x  john  Documents
-rw-r--r--  john  notes.txt
```

```
$ ls -la                                # Long listing + hidden files
(starting with .)
drwxr-xr-x  john  .
drwxr-xr-x  root  ..
-rw-r--r--  john  .bashrc

$ cd Documents                          # Change into the Documents directory
$ cd ..                                # Go up one level (to parent
directory)
$ cd ~                                  # Go to your home directory from
anywhere
$ cd /etc                               # Go to an absolute path (starting
from /)
```

TIP Absolute paths start with / (from root). Relative paths start from where you are. For example: `cd ../Downloads` is relative, `cd /home/john/Downloads` is absolute.

2.3 Creating Files and Directories

```
$ mkdir projects                        # Create a new directory
$ mkdir -p projects/web/css            # Create nested directories at once (-p flag)
$ touch index.html                     # Create an empty file
$ touch file1.txt file2.txt            # Create multiple files at once
```

2.4 Copying, Moving, and Deleting

```
# Copying files and directories
$ cp notes.txt backup.txt              # Copy file to new name
$ cp notes.txt Documents/              # Copy file into a directory
$ cp -r projects/ projects_backup/    # Copy directory and all contents (-r =
recursive)

# Moving and renaming
$ mv backup.txt Documents/             # Move file into a directory
$ mv notes.txt readme.txt              # Rename a file (move to new name)
```

```
# Deleting
$ rm oldfile.txt           # Delete a single file
$ rm file1.txt file2.txt   # Delete multiple files
$ rm -r old_project/       # Delete a directory and all its contents
$ rm -rf build/            # Force delete — no confirmation prompt
```

WARN `rm` deletes permanently — there is no Recycle Bin in Linux. The `-rf` flag is especially dangerous. Always double-check the path before running `rm -rf`.

2.5 Viewing File Content

Command	Description
<code>cat file.txt</code>	Print the entire file content to the terminal
<code>less file.txt</code>	View file page by page. Use arrows to scroll, Q to quit
<code>head -5 file.txt</code>	Show only the first 5 lines of a file
<code>tail -10 file.txt</code>	Show only the last 10 lines of a file
<code>tail -f log.txt</code>	Follow a file live — useful for watching log files
<code>wc -l file.txt</code>	Count the number of lines in a file
<code>grep 'word' file.txt</code>	Search for 'word' inside a file — shows matching lines
<code>find . -name '*.txt'</code>	Find all .txt files starting from current directory

3. Permissions and Ownership

Linux is a multi-user system. Every file and directory has an owner and a set of permissions that control who can read, write, or execute it. Understanding permissions is essential for security and proper system administration.

3.1 Reading the Permission String

When you run `ls -l`, the first column shows a 10-character permission string. Here is how to read it:

```
$ ls -l
-rwxr-xr-- 1 john  staff  1024  Apr 11  script.sh
|   |   |
|   |   └─ Others (everyone else): r-- = read only
|   └───── Group (staff):          r-x = read + execute
└────────── Owner (john):          rwx = read + write + execute
```

The first character: - = regular file, d = directory, l = symbolic link

The three permission types and their numeric values:

Symbol	Permission	Numeric value	Meaning
r	Read	4	Open and view the file/directory
w	Write	2	Modify or delete the file/directory
x	Execute	1	Run the file or enter the directory
-	None	0	No permission granted

3.2 chmod — Changing Permissions

The `chmod` command (change mode) modifies file permissions. There are two ways to use it: symbolic (using letters) and numeric (using numbers).

Symbolic method (easier to read):

```
# Format: chmod [who][operator][permission] filename
# who:      u = user/owner,  g = group,  o = others,  a = all
# operator: + = add,        - = remove,  = = set exactly
```

```
$ chmod u+x script.sh    # Give the owner execute permission
$ chmod g-w file.txt     # Remove write permission from group
$ chmod o+r file.txt     # Give others read permission
```

```
$ chmod a+x script.sh      # Give everyone execute permission
$ chmod go-w private.txt # Remove write from group AND others
```

Numeric (octal) method:

```
# Each permission set is a 3-digit number (owner, group, others)
# r=4, w=2, x=1. Add them up for each set.
#
# 7 = 4+2+1 = rwx    (full access)
# 6 = 4+2+0 = rw-    (read and write)
# 5 = 4+0+1 = r-x    (read and execute)
# 4 = 4+0+0 = r--    (read only)

$ chmod 755 script.sh      # rwxr-xr-x : run by all, writable only by owner
$ chmod 644 file.txt       # rw-r--r-- : most common for regular files
$ chmod 600 secret.txt     # rw----- : only owner can read/write
$ chmod 777 shared.sh      # rwxrwxrwx : everyone has full access (use carefully!)
```

TIP 755 and 644 are the most commonly used permissions. Use 755 for scripts and programs, 644 for regular text files and documents.

3.3 chown — Changing Ownership

The `chown` command (change owner) transfers ownership of a file from one user or group to another. You usually need `sudo` (admin) privileges to change ownership to another user.

```
$ chown john file.txt      # Change file owner to 'john'
$ chown john:staff file.txt # Change owner AND group
$ sudo chown root config.cfg # Transfer to root (needs sudo)
$ sudo chown -R john mydir/  # Recursively change owner of all files in a dir

# View current ownership
$ ls -l file.txt
-rw-r--r-- 1 john staff 1024 Apr 11 file.txt
#           ↑      ↑
#           owner  group
```

3.4 sudo — Running as Administrator

`sudo` (superuser do) lets you run a command with administrator (root) privileges. It is needed for installing software, changing system files, or managing other users.

```
$ sudo apt update      # Update the package list (needs admin)
$ sudo apt install vim  # Install a software package
$ sudo apt upgrade     # Upgrade all installed packages
```



```
# sudo asks for YOUR password (not a separate root password)
$ sudo nano /etc/hosts          # Edit a system configuration file
```

WARN Never run `sudo rm -rf /` or `sudo rm -rf /*` — this would permanently delete your entire Linux system. Always verify the path when using `sudo` with `rm`.

4. Processes and System Information

Every running program in Linux is called a process. Each process has a unique PID (Process ID). Linux is a multitasking system — it runs dozens or hundreds of processes simultaneously, all managed by the kernel.

4.1 Viewing Processes

```
$ ps                                     # Show processes in the current terminal session
  PID TTY          TIME CMD
 1024 pts/0    00:00:00 bash
 1087 pts/0    00:00:00 ps

$ ps aux                                # Show ALL processes on the system
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   COMMAND
root         1   0.0  0.1 169000 13000 ?        Ss   10:00   /sbin/init
john    1024   0.0  0.2   21000   5000 pts/0    Ss   10:01   bash
john    1087   2.1  0.5  712000 11000 pts/0    Sl   10:05   python app.py

$ pgrep bash                             # Find the PID of a running process by name
1024
```

4.2 Live Process Monitor — top and htop

The **top** command gives a real-time, updating view of all running processes. It is like the Task Manager in Windows.

```
$ top                                     # Launch the live process monitor

top - 10:30 up 2 days, load average: 0.15, 0.12, 0.10
Tasks: 120 total,  1 running, 119 sleeping
%Cpu(s):  2.1 us,  0.5 sy,  97.2 id
MiB Mem : 16000.0 total,  8000.0 free,  4000.0 used

Keyboard shortcuts inside top:
 k    → Kill a process (you will be asked for the PID)
 M    → Sort by memory usage
 P    → Sort by CPU usage
 q    → Quit
```

htop is a modern, colorful alternative to **top** with mouse support. Install it first:

```
$ sudo apt install htop                 # Install htop
$ htop                                  # Launch the interactive process viewer
```

4.3 Managing Processes

```
# Killing processes
$ kill 1087          # Send SIGTERM (polite stop) to PID 1087
$ kill -9 1087       # Send SIGKILL (force stop) – use as last resort
$ killall firefox    # Kill all processes named firefox

# Background and foreground jobs
$ sleep 60 &         # Run command in background (&)
[1] 2045              # Job number [1] and PID 2045 are shown

$ jobs               # List all background jobs
[1]+  Running      sleep 60 &

$ fg %1              # Bring job [1] to foreground
$ bg %1              # Resume a paused job in the background

# Press Ctrl+C to stop the current foreground process
# Press Ctrl+Z to pause it and send it to the background
```

4.4 System Information Commands

Command	Description
<code>free -h</code>	Show total, used, and free RAM in human-readable units (MB/GB)
<code>df -h</code>	Show disk space usage for each mounted partition
<code>du -sh *</code>	Show the size of each file and folder in the current directory
<code>uptime</code>	Show how long the system has been running and the CPU load average
<code>lscpu</code>	Show detailed CPU information — cores, speed, architecture
<code>lsblk</code>	Show all storage devices (disks, partitions, USB drives)
<code>uname -r</code>	Show the current Linux kernel version
<code>cat /etc/os-release</code>	Show the Linux distribution name and version

5. Redirection and Pipes

Redirection and pipes are among the most powerful features of Unix/Linux. They let you control where data comes from and where it goes — turning the terminal into a data processing pipeline.

5.1 Standard Streams

Every Linux process has three open communication channels called standard streams:

Command	Description
<code>stdin (0)</code>	Standard Input — the data coming IN to a program. Default: keyboard.
<code>stdout (1)</code>	Standard Output — the data going OUT from a program. Default: screen.
<code>stderr (2)</code>	Standard Error — error messages from a program. Default: screen.

Redirection lets you change where these streams point — sending output to a file, or reading input from a file instead of the keyboard.

5.2 Output Redirection

```
# > overwrites the file (creates it if it doesn't exist)
$ ls > filelist.txt           # Save the output of ls to a file
$ date > timestamp.txt       # Save current date to a file

# >> appends to the file (adds to the end without erasing)
$ date >> logfile.txt         # Add current date to the end of logfile.txt
$ echo 'Done' >> logfile.txt  # Add a message to the log

# View the result
$ cat logfile.txt
Sat Apr 11 10:30:00 IST 2026
Done
```

5.3 Input Redirection and Error Redirection

```
# < sends a file as input to a command
$ sort < names.txt           # Sort the contents of names.txt

# 2> redirects error messages to a file
$ ls /fakepath 2> errors.txt # Save error messages to errors.txt
```

```
$ ls /fakepath 2>/dev/null      # Discard errors completely (/dev/null is a black
hole)

# Redirect both stdout and stderr to the same file
$ ls /fakepath > output.txt 2>&1
# 2>&1 means 'send stderr (2) to wherever stdout (1) is going'
```

5.4 Pipes — The Unix Superpower

The pipe character `|` sends the output of one command directly as the input to the next command. This lets you chain multiple tools together to process data step by step — without creating any temporary files.

Think of it like an assembly line: each command is a station that does one job and passes the result to the next.

```
# Basic pipe examples
$ ls -la | grep '.txt'          # List files, then filter to show only .txt files
$ ps aux | grep firefox        # Show all processes, filter to find firefox
$ cat /etc/passwd | wc -l       # Count the number of user accounts
$ history | tail -20            # Show your last 20 commands

# Chaining multiple pipes
$ cat names.txt | sort | uniq   # Read file, sort lines, remove duplicates

$ cat access.log | grep 'ERROR' | wc -l
# Count how many ERROR lines are in access.log

$ df -h | grep '/dev/sd'        # Show only physical disk partitions
```

TIP You can chain as many pipes as you need. Each `|` connects the stdout of the left command to the stdin of the right command. The more you use pipes, the more you think in the Unix philosophy.

5.5 Useful Commands for Pipelines

Command	Description
<code>grep 'pattern'</code>	Filter lines matching a pattern — the most used pipe command
<code>sort</code>	Sort lines alphabetically or numerically
<code>uniq</code>	Remove consecutive duplicate lines (use after sort)
<code>wc -l</code>	Count lines; <code>-w</code> counts words, <code>-c</code> counts characters
<code>head -n</code>	Keep only the first <code>n</code> lines

Command	Description
<code>tail -n</code>	Keep only the last n lines
<code>cut -d: -f1</code>	Extract specific columns (split by delimiter ':')
<code>tr 'a-z' 'A-Z'</code>	Translate characters — here: lowercase to uppercase

6. Shell Scripting Basics

A shell script is a plain text file containing a sequence of Linux commands. Instead of typing them one by one every time, you write them once and run the file. Scripts are the foundation of automation in Linux.

6.1 Creating and Running Your First Script

```
# Step 1: Create the script file
$ nano myscript.sh           # Opens the nano text editor

# Step 2: Make it executable
$ chmod +x myscript.sh

# Step 3: Run it
$ ./myscript.sh              # ./ means 'in the current directory'
```

TIP The .sh extension is just a convention — it tells humans the file is a shell script. Linux does not require file extensions. What matters is the shebang line inside the file.

6.2 The Shebang Line

Every shell script should start with a shebang line. This tells the operating system which interpreter to use to run the script.

```
#!/bin/bash

# This is a comment — the # symbol starts a comment
# Comments are ignored by the shell but essential for documentation

echo "This is my first shell script!"
```

6.3 Variables

Variables store data that your script can use. In bash, there are no spaces around the = sign when assigning a value.

```
#!/bin/bash

# Defining variables — NO spaces around the = sign
NAME="Alice"
AGE=22
CITY="Chennai"
```

```
# Using variables – prefix with $
echo "Hello, $NAME!"
echo "You are $AGE years old and live in $CITY."

# Capturing command output into a variable
TODAY=$(date +%Y-%m-%d)      # Runs the date command and stores result
echo "Today is: $TODAY"

# Special variables
echo "Script name: $0"        # $0 = name of the script
echo "First argument: $1"     # $1 = first argument passed to the script
echo "All arguments: $@"      # $@ = all arguments
echo "Number of args: $#"     # $# = count of arguments
```

6.4 Reading User Input

```
#!/bin/bash

echo -n "Enter your name: "    # -n prevents a newline after the prompt
read USER_NAME

echo "Welcome, $USER_NAME!"

# Read with a prompt in one line
read -p "Enter your age: " USER_AGE
echo "You are $USER_AGE years old."
```

6.5 Conditional Statements (if / else)

```
#!/bin/bash

read -p "Enter a number: " NUM

if [ $NUM -gt 0 ]; then
    echo "The number is positive."
elif [ $NUM -lt 0 ]; then
    echo "The number is negative."
else
    echo "The number is zero."
fi                                # fi ends the if block (if reversed)
```



```
# String comparison
read -p "Enter your city: " CITY

if [ "$CITY" == "Chennai" ]; then
    echo "Weather: Very hot and humid!"
else
    echo "Weather: Check the forecast."
fi
```

Command	Description
-eq	Equal to (numbers)
-ne	Not equal to (numbers)
-gt	Greater than
-lt	Less than
-ge	Greater than or equal to
==	Equal to (strings)
!=	Not equal to (strings)
-f file	True if file exists and is a regular file
-d dir	True if directory exists

6.6 Loops

For loop:

```
#!/bin/bash

# Loop over a list of values
for i in 1 2 3 4 5; do
    echo "Count: $i"
done

# Loop over files in a directory
for file in *.txt; do
    echo "Processing: $file"
    wc -l "$file"
done

# C-style for loop
for ((i=1; i<=5; i++)); do
    echo "Number: $i"
done
```

While loop:

```
#!/bin/bash

count=1
while [ $count -le 5 ]; do
    echo "Line number: $count"
    count=$((count + 1))      # Arithmetic uses $(( ))
done
```

6.7 Functions

Functions let you group commands under a name so you can reuse them. This keeps scripts clean and avoids repetition.

```
#!/bin/bash

# Define a function
greet() {
    echo "Hello, $1!"          # $1 = first argument passed to the function
    echo "Welcome to Linux."
}

# Call the function
greet "Alice"
greet "Bob"

# Function with return value
add_numbers() {
    local result=$(( $1 + $2 )) # local makes the variable function-scoped
    echo $result
}

SUM=$(add_numbers 10 25)
echo "10 + 25 = $SUM"
```

6.8 Practical Example — Automated Backup Script

This is a complete, practical script that backs up a folder with a date-stamped name. This is the kind of automation that system administrators use every day.

```
#!/bin/bash
#
# backup.sh - Backs up a directory with a timestamp
```

```
# Usage: ./backup.sh
#

SOURCE="/home/john/Documents"
DEST="/home/john/backups"
DATE=$(date +%Y-%m-%d_%H-%M)
BACKUP_NAME="docs_$(DATE)"

# Create the backup destination if it does not exist
mkdir -p $DEST

# Check that the source folder exists
if [ ! -d "$SOURCE" ]; then
    echo "Error: Source directory $SOURCE not found!"
    exit 1 # Exit with error code 1
fi

# Perform the backup
cp -r $SOURCE $DEST/$BACKUP_NAME

# Report success
echo "Backup complete!"
echo "Saved to: $DEST/$BACKUP_NAME"
```

TIP To schedule this script to run automatically every night, use cron: run 'crontab -e' and add a line like: 0 2 * * * /home/john/backup.sh (runs at 2:00 AM daily)

Quick Reference — Essential Commands

Navigation & Files

Command	Description
<code>pwd</code>	Print current directory path
<code>ls -la</code>	List all files with details and hidden files
<code>cd dirname</code>	Change into a directory
<code>cd ..</code>	Go up one directory level
<code>mkdir -p a/b/c</code>	Create nested directories
<code>touch file.txt</code>	Create an empty file
<code>cp -r src/ dst/</code>	Copy directory and contents recursively
<code>mv file dest/</code>	Move or rename a file
<code>rm -rf dir/</code>	Delete directory permanently (use with caution)
<code>find . -name '*.sh'</code>	Find files by name pattern

Permissions

Command	Description
<code>ls -l</code>	Show file permissions
<code>chmod 755 file</code>	Set permissions numerically (rwxr-xr-x)
<code>chmod u+x script.sh</code>	Add execute for owner
<code>chown user:group file</code>	Change owner and group
<code>sudo command</code>	Run command as administrator

Processes

Command	Description
<code>ps aux</code>	Show all running processes
<code>top / htop</code>	Live process monitor
<code>kill PID</code>	Terminate a process by PID
<code>kill -9 PID</code>	Force kill a process
<code>command &</code>	Run command in background
<code>jobs</code>	List background jobs
<code>fg %1</code>	Bring background job 1 to foreground

Redirection & Pipes

Command	Description
<code>cmd > file</code>	Redirect output to file (overwrite)
<code>cmd >> file</code>	Redirect output to file (append)
<code>cmd 2>/dev/null</code>	Discard error messages
<code>cmd1 cmd2</code>	Pipe output of cmd1 to cmd2
<code>grep 'text' file</code>	Search for text in a file
<code>sort file uniq</code>	Sort and remove duplicates